Ruby lets you specify default values for a method's parameters—values that will be used if the caller doesn't pass them explicitly. You do this using an equal sign (=) followed by a Ruby expression. That expression can include references to previous parameters in the list:

```
def cool_dude(arg1="Miles", arg2="Coltrane", arg3="Roach")
  "#{arg1}, #{arg2}, #{arg3}."
end

cool_dude                           # => "Miles, Coltrane, Roach."
cool_dude("Bart")                   # => "Bart, Coltrane, Roach."
cool_dude("Bart", "Elwood")         # => "Bart, Elwood, Roach."
cool_dude("Bart", "Elwood", "Linus") # => "Bart, Elwood, Linus."
```

Here's an example where the default parameter references a previous parameter:

```
def surround(word, pad_width=word.length/2)
  "[" * pad_width  + word +  "]" * pad_width
end

surround("elephant") # => "[[[[elephant]]]]"
surround("fox")      # => "[fox]"
surround("fox", 10)  # => "[[[[[[[[[[fox]]]]]]]]]]"
```

The default parameter value is re-evaluated every time the method is called, and any variable that would be visible inside the method itself is available for the default value expression.

Variable-Length Parameter Lists

But what if you want to pass in a variable number of parameters or want to capture multiple arguments into a single parameter? Placing an asterisk before the name of the parameter lets you do just that. This is sometimes called a *splat* (presumably because the asterisk looks somewhat like a bug after hitting the windscreen of a fast-moving car).

```
def variable_args(arg1, *rest)
  "arg1=#{arg1} -- rest=#{rest.inspect}"
end

variable_args("one")                    # => arg1=one -- rest=[]
variable_args("one", "two")             # => arg1=one -- rest=["two"]
variable_args("one", "two", "three")    # => arg1=one -- rest=["two", "three"]
```

In this example, the first argument is assigned to the first method parameter as usual. However, the next parameter is prefixed with an asterisk, so all the remaining arguments are bundled into a new Array, which is then assigned to that parameter.

Folks sometimes use a splat to specify parameters that are not used by the method but that are perhaps used by the corresponding method in a superclass. (Note that in this example we call super with no parameters. This is a special case that means "invoke this method in the superclass, passing it all the parameters that were given to the original method.", more about this in Super Lookup, on page 114.)

```
class Child < Parent
  def do_something(*not_used)
    # our processing
    super
  end
end
```